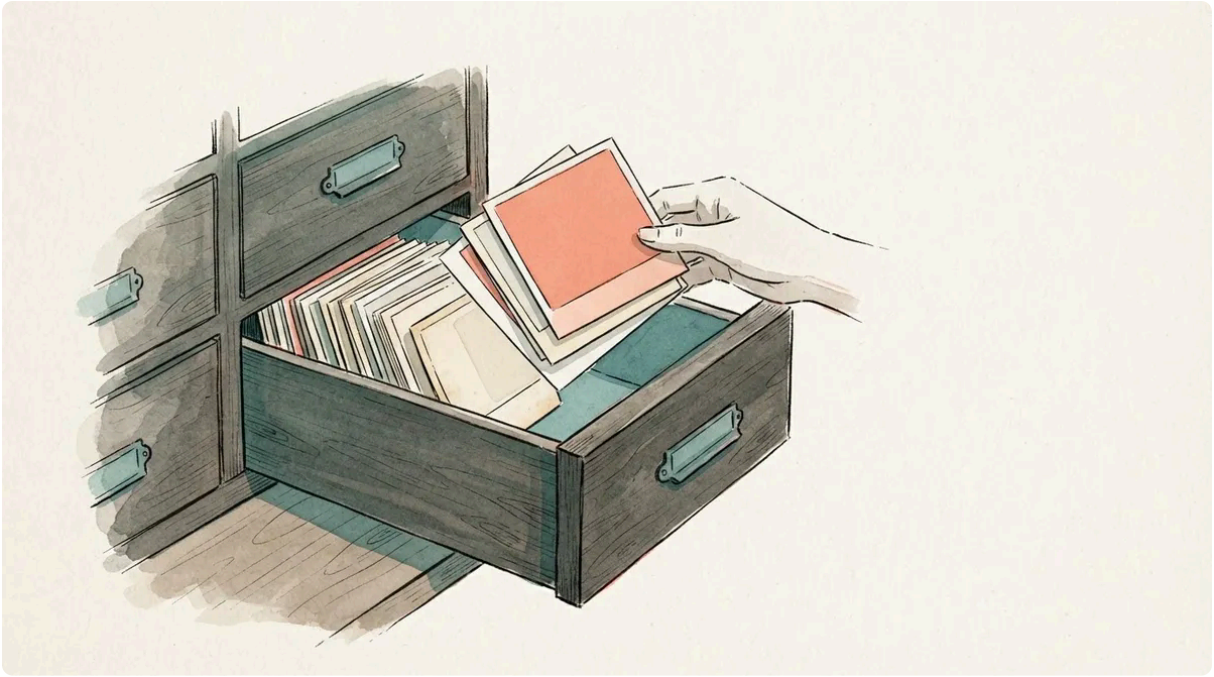
Images on a static site: tradeoffs, limits, and a 150-line Rust optimizer

Emil Lindfors | February 23, 2026



Images on this blog live in git, in the same directory as the post that uses them, as WebP files that a small Rust tool made from whatever I had. That was the decision in February, when the site had no pictures at all and I was about to write about sensor rigs and salmon farms. Six months on the decision has held, the tool has grown, and the pictures turned out to be something I had not planned for. This is the setup, the arithmetic behind it, and what changed.

The options

There are three reasonable places to put images on a static site deployed to Cloudflare Pages.

Next to the post, in git. The image sits in the post's directory and Zola copies it to the output. Version-controlled, no extra service, and a link like `sensor-rig.webp` in the markdown.

Object storage. Cloudflare R2, S3, anything with a bucket. The repo stays small. You gain an upload step, URL management and one more service to configure.

Inline as base64. Encode the image into the HTML. No extra files to serve, and a terrible idea in practice: 33% bigger from the encoding, no browser caching (the image is downloaded again with every page), and more HTML to parse before anything renders.

Why git

Everything on this site is one repo. The fonts are self-hosted, the newsletter Worker is checked in beside the templates, and the PDF pipeline reads the same markdown. A second deployment target for images would have been the first thing that lived somewhere else.

The practical question is whether git can carry it. For a blog, easily.

The math

A well-optimised blog image, WebP at 1200 pixels wide and quality 80, is 30 to 150 KB. A diagram as SVG is often under 10 KB. At 150 KB an image, here is how far you get:

Images	Repo size	Git performance
100	~15 MB	No impact
500	~75 MB	No impact
1,000	~150 MB	Fine
2,000	~300 MB	Still fine

Git starts feeling slow somewhere around 500 MB to 1 GB of binary content. That is 3,000 to 7,000 optimised images. I am not going to write 3,000 posts.

Cloudflare Pages allows 20,000 files per deployment and 25 MB per file. Neither is a concern for images.

When object storage makes sense

Video, large downloadable datasets, or galleries with dozens of full-resolution photographs per post. Then R2 on a subdomain is the right move, and the migration is changing image paths from relative to absolute. That is a bridge to cross when I reach it.

The tool

The key to keeping images in git is optimising before committing. A 4 MB photograph from a camera has no business in a repository or in a browser. What you want is a WebP at a sensible width.

I wrote a small Rust CLI for it, `img-optim`. Two crates do the work: `image` for decoding and resizing, and `webp`, which wraps libwebp, for lossy encoding with a quality setting.

Cargo.toml

```
[package]
name = "img-optim"
version = "0.1.0"
edition = "2021"

[dependencies]
image = { version = "0.25", default-features = false, features = [
    "jpeg", "png", "gif", "bmp", "tiff"
] }
webp = "0.3"

[profile.release]
opt-level = 3
```

One thing to note here. The `image` crate's default features include a pure-Rust WebP encoder that only does lossless. For lossy with quality control you need the `webp` crate, which links against libwebp. On Debian and Ubuntu that is `apt install libwebp-dev`.

The core logic

The interesting part is three functions, about 150 lines with the argument parsing:

```
fn resize_to_width(img: image::DynamicImage, max_width: u32) ->
image::DynamicImage {
    let (w, h) = img.dimensions();
    if w > max_width {
        let new_h = (max_width as f64 / w as f64 * h as f64) as u32;
        img.resize_exact(max_width, new_h,
image::imageops::FilterType::Lanczos3)
    } else {
        img
    }
}

fn encode_webp(
    img: &image::DynamicImage,
    path: &Path,
    quality: f32,
) -> Result<(), Box<dyn std::error::Error>> {
    let encoder = webp::Encoder::from_image(img)
        .map_err(|e| format!("webp encode: {e}"))?;
```

```
    let data = encoder.encode(quality);
    fs::write(path, &*data)?;
    Ok(())
}

fn optimize(
    path: &Path,
    max_width: u32,
    quality: f32,
) -> Result<(u64, u64, PathBuf), Box<dyn std::error::Error>> {
    let before = fs::metadata(path)?.len();
    let img = image::open(path)?;
    let img = resize_to_width(img, max_width);

    let out_path = path.with_extension("webp");
    encode_webp(&img, &out_path, quality)?;

    let after = fs::metadata(&out_path)?.len();
    Ok((before, after, out_path))
}
```

`resize_to_width` uses Lanczos3 resampling. It keeps edges sharper than bilinear or bicubic when downscaling a photograph, and it does nothing when the image is already narrow enough.

`encode_webp` wraps libwebp. Quality 80 is the default. At blog sizes I cannot tell it from the original, and the file is a fraction of the size.

Thumbnails

The featured cards on the front page do not need 1200 pixels. A 600-pixel thumbnail at slightly lower quality is plenty for a card. The `-t` flag makes one:

```
const THUMB_WIDTH: u32 = 600;
const THUMB_QUALITY: f32 = 75.0;
const THUMB_SUFFIX: &str = "-thumb";

fn thumbnail(
    path: &Path,
    width: u32,
    quality: f32,
) -> Result<(u64, PathBuf), Box<dyn std::error::Error>> {
    let img = image::open(path)?;
    let img = resize_to_width(img, width);

    let stem = path.file_stem().unwrap().to_string_lossy();
    let out_path = path.with_file_name(format!("{stem}{THUMB_SUFFIX}.webp"));
```

```
    encode_webp(&img, &out_path, quality)?;

    let size = fs::metadata(&out_path)?.len();
    Ok((size, out_path))
}
```

The names are fixed: `hero.jpg` becomes `hero.webp` and `hero-thumb.webp`. The templates derive the thumbnail path from `featured_image` with a string replace, so the frontmatter names one file.

Results

The two test images I ran it on in February:

```
$ img-optim -t photo.jpg diagram.png
  photo.jpg → photo.webp (47.7 KB → 22.3 KB, -53%)
  photo.jpg → photo-thumb.webp (17.3 KB)
  diagram.png → diagram.webp (261.6 KB → 7.5 KB, -97%)
  diagram.png → diagram-thumb.webp (2.6 KB)

  Total: 309.3 KB → 49.7 KB (-84%)
```

And a real one from this week, the hero at the top of this post as it came out of an image model:

```
  hero.jpg → hero.webp (175.2 KB → 67.7 KB, -61%)
  hero.jpg → hero-thumb.webp (16.4 KB)
```

PNG diagrams compress spectacularly. Photographs and illustrations get roughly halved by the format change, the quality setting and the width cap together.

The workflow

Adding images to a post:

```
content/blog/my-post/
├── index.md
├── hero.webp          # featured image (1200px, q80)
├── hero-thumb.webp    # thumbnail for the front page (600px, q75)
├── sensor-rig.webp    # inline image
└── results-chart.svg  # diagrams stay as SVG
```

The frontmatter:

```
[extra]
featured_image = "hero.webp"
```

In the markdown, relative paths:

```
![The sensor rig mounted on the cage](sensor-rig.webp)
```

The commands:

```
# Drop raw images into the post directory, then:
img-optim -t content/blog/my-post/hero.jpg
img-optim content/blog/my-post/sensor-rig.jpg

# Delete the originals, commit the .webp files
rm content/blog/my-post/*.jpg
```

`-t` is only for the featured image. Inline images do not need thumbnails.

What the template does

`featured_image` drives four things:

- **The post header.** The title and description sit on the picture, under a dark gradient, so the article starts one band sooner than it did when the picture came after the title.
- **The front page.** Featured cards get the same treatment, the lead card with the full hero and the other two with their thumbnails.
- **The PDF.** The hero appears on the first page.
- **Social sharing.** Every post has a 1200x630 card at `/og/<slug>.png`. With a hero and no better card, the title is composed over the hero. How the better cards are made is its own post.

A post without `featured_image` renders exactly as before. Text only, no placeholder, no broken layout.

Format recommendations

WebP for photographs, illustrations and screenshots. Lossy at quality 80 is the sweet spot. Every browser you care about supports it.

SVG for diagrams, charts and anything with clean lines and text. Scales without limit, tiny, and it follows the site's light and dark themes if you use `currentColor`.

Not PNG for photographs. Lossless means large files for no visible gain. PNG is fine for screenshots where pixel-perfect text matters, and WebP lossless does that too, smaller.

Not JPEG as the final format. WebP at the same visual quality is 25 to 35% smaller. Use JPEG as the source you convert from.

Six months on

Two things did not go the way the February version of this post expected, and one did.

No photograph ever arrived. Every picture on this site today is a hero image, and every one of them is generated. The sensor rigs and the salmon farms are still in my head. The tool that was built for DSLR photographs has converted only illustrations, which it does not know or care about.

The tool grew. It is 531 lines now, with tests, up from 325. A `-q` given as the last argument used to panic by indexing past the end of argv, so the argument parsing returns errors instead. Quality is bounded to 0 to 100, because libwebp clamps silently: `-q 900` encoded at 100 and looked like it had worked. Animated GIFs convert to animated WebP. And the build refuses to run with an unconverted JPEG or PNG under `content/`, because `deploy.sh` runs `git add -A`, and a forgotten 4 MB source would be in the history for good. That check has already caught one file, a card that the generation tool wrote as a JPEG.

Git is fine. Thirteen posts with heroes, thumbnails and cards come to under 2 MB of images. The table above still has three zeros to spare.

A note on the hero image

The hero on this post was originally generated with Qwen-Image-2.0, Alibaba's open model from February 2026, as a decorative anchor for a post about an abstract topic. In September it was replaced with the drawer of prints at the top of the page, drawn by a different model in the style every post now shares. It went through `img-optim` like anything else: 175 KB of JPEG to a 68 KB WebP with a 16 KB thumbnail. The same picture is now the style reference that every other hero on the site is drawn against, which is the next post.